Compiler Construction: Project Phase 3

Due on **Friday, November 6, 2026 at 11:59 P.M.**

1 Description

Your task for this project is to build a static semantic analyzer that augments the symbol table and Abstract Syntax Tree (AST) (built by the parser) with appropriate semantic information. The semantic analyzer will utilize the symbol table and AST to detect errors in the program related to scoping rules and type mismatch.

2 Extending the Symbol Table

In this phase, you need to classify the identifiers into two categories: identifiers that refer to variables and identifiers that refer to functions. For each variable the symbol table should contain three types of information: name, type and scope. For function identifiers, the type is represented as the functions signature. The function signature includes the return type, the number of parameters, and the type of each parameter.

3 Types in `mC`

A variable identifier in `mC` can have the following basic types: `int`, `char` or `void`. A variable identifier can also be an array of each of the basic types. For this assignment you will not do any type promotion, type checking is only based on whether the types provided are the same.
For example, if `int i = 5` and `char c = 'z'` then the types of `i` and `c` are considered different.

4 Scoping Rules in `mC`

There are only two types of scope possible for a variable identifier: global and local to a function. Any variable declared outside a function is considered global, and any variable declared within a function is local to that function. NOTE, you can have multiple variables with the same name, e.g., `int i`, declared locally within different functions.

- Global scope
  All functions are declared within the global scope
  A function-local scope begins with each function declaration
- Function-local scopes
  All function arguments are variables local to the function
  All variables declared within a function are local to the function
  Name lookups check the local scope before the global scope
  (Local variables shadow global variables with the same name)
  The scope is reset to global following the end of a function body

5 Semantic Error Checking

The semantic analyzer adds detection of the following errors:

1. Use of undeclared identifiers
   A variable or function is considered undeclared if it is not found in the function scope (if within a function-local scope) or the global scope.
2. Multiply declared identifiers
   A variable or function is considered multiply declared if it has more than one declaration in the current scope. NOTE, two declarations, e.g., `int a` and `char a`, within the same scope are considered multiply defined as they have the same *name*.
3. Function definition/call mismatch
   A function definition includes zero or more arguments
   A mismatched function call uses the wrong number of arguments or wrong type of any argument.
4. Array indexing
   Only variables defined as an array with a [*size*] should be indexed as an array
   Arrays should not be indexed with a non-integer or an out-of-bounds integer
5. Type mismatch in assignments
   If the left-hand and right-hand sides of an assignment statement are of different types then your semantic analyzer should generate an error message.
   For example, trying to assign a `char` to an `int`.
   NOTE: this check only needs to be performed when the right-hand side is a simple expression, i.e., a variable, literal, or a function call.

6 Extra Credit: AST Minimization

As an optional extra credit opportunity, you may extend your semantic analyzer to minimize the AST. This involves removing or collapsing redundant nodes from the tree after semantic analysis — for example, eliminating nodes that carry no semantic meaning (such as pass-through productions with a single child). Minimizing the AST is **not** a strict requirement for this assignment and will not affect your base score.

7 Compilation and Testing

- A subset of the test cases which will be used are provided for you to test your code, think about edge-cases where your program may fail – testing your code is equally as important as writing the code itself
- Your program will be tested on the CS department Linux servers (`zeus` or `eros`). Make sure it builds and runs correctly there before you submit
- Unless indicated in `writeup.txt`, your program will be compiled using the provided `Makefile` with the commands:
  `make clean ; make`
- Please ensure the latest version of all files are present in your submission

8 Writeup

You need to write a short document named `writeup.txt`.
Within `writeup.txt` you should include the following (numbered) sections:

1. Your name and course ID
2. Group members and their respective contributions
3. Provide two examples different from those provided in this document of how you will test the correctness of your code
4. Extra instructions if you do not use the provided `Makefile`

9 Submission

The project may be done individually or as a group of two. Please submit on CANVAS a ZIP archive named "ProjectPhase3_<netID1>_<netID2>.zip" containing your files. Ensure the directory structure is exactly as we have provided. Only what is submitted on Canvas is graded: a link to your repository, or a commit pushed after the deadline, does not count. The grading rubric is provided in Table 1.

Late submission

Each project group has three (3) slip days for the semester, shared across all four phases. A slip day extends this deadline by 24 hours, applies to the entire group submission, and must be declared at submission time. Slip days are never applied automatically.

Declared slip days are subtracted from the number of days late first, and whatever remains is penalized as in Table 2. Any portion of a day late counts as one full day late. After slip days are subtracted, a submission more than three days late is not accepted.

Category	Points
Symbol Table (type, scope, function info)	20
Undeclared/Multiply Declared Variables	10
Undeclared/Multiply Declared Functions	10
Function Call Mismatch	10
Array Indexing	10
Type Mismatch	10
Compile & Run	10
Documentation	10
Writeup	10

Table 1: Rubric for Project Phase 3

Days Late (after slip days)	Points Deducted
1 Day	15%
2 Days	30%
3 Days	45%
> 3 Days	100% (no credit)

Table 2: Late Submission Policy